

Revy(원량훈) 포트폴리오 - Core Banking

DEMO 주소 정보

- WEB-ADMIN : <https://demo-admin.revy-won.dev/>
- WEB-SAAS : <https://demo.revy-won.dev/>
- Github(포트폴리오 소스) : <https://github.com/wantedhooni/portfolio/tree/develop/core-banking>
- Github : <https://github.com/wantedhooni/>
- blog : <https://revy.notion.site/Revy-Logs-or-audit-2cf8f9e43b5f8034a60ff8b40019c0ca>

1. 프로젝트 개요

Revy Core Banking은 인터넷전문은행을 가정한 풀스택 코어뱅킹 포트폴리오 프로젝트다. 백엔드는 Spring Boot 멀티모듈 구조로 계좌, 거래, 주식, 주문, 포트폴리오, 외환, 보험, 원장, 청구, 정산, 스케줄러를 분리했고, 프론트엔드는 관리자 콘솔과 사용자 SaaS 화면을 Next.js로 분리했다.

구분	내용
백엔드	Java 25, Spring Boot 4, Gradle 멀티모듈, JPA, QueryDSL, Flyway, Spring Security, JWT, Redis
프론트엔드	Next.js 16, React 19, TypeScript, Tailwind CSS, shadcn/ui, Axios
데이터베이스	PostgreSQL, Flyway migration
캐시/세션	Redis refresh token 저장
앱 구성	api-admin , api-saas , server-executor , web-admin , web-saas
운영 스택	Pgpool-II, PostgreSQL Primary/Secondary, Redis, ELK, Prometheus/Grafana

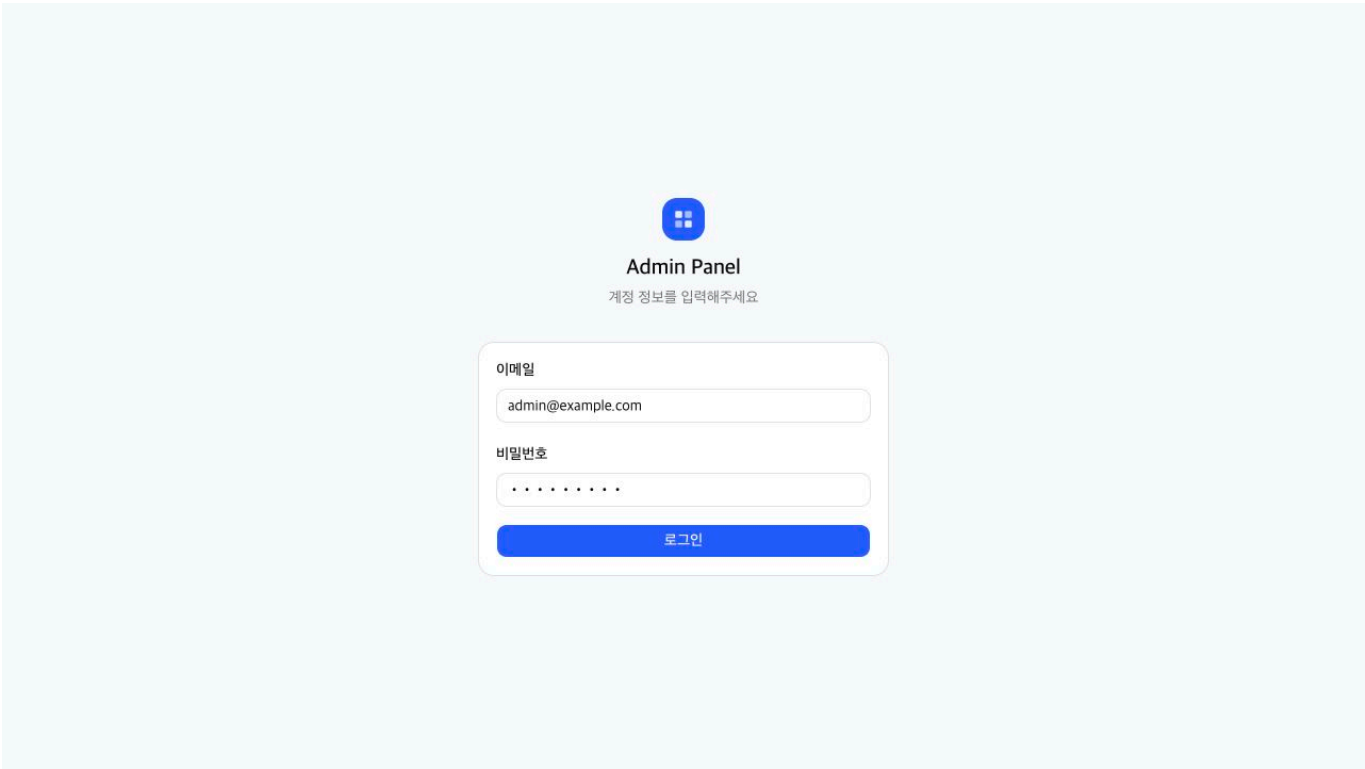
최근 작업에서 중점적으로 본 부분

이번 업데이트에서는 “기능이 많다”보다 “운영 가능한 금융 시스템처럼 책임이 나뉘어 있는가”를 기준으로 다시 정리했다. 외환은 현재 환율만 저장하는 구조에서 환율 이력과 통화 회랑까지 분리했고, 스케줄러는 관리자 API가 직접 실행까지 말지 않도록 **server-executor**를 별도 워커로 뺐다. 이 두 지점은 포트폴리오에서 단순 CRUD를 넘어 운영 통제, 감사 추적, 실행 책임 분리를 보여주는 부분이다.

프론트엔드 화면 - web-admin

운영자 콘솔은 도메인별 백오피스 업무를 서버사이드 검색, 페이지네이션, 등록/수정/삭제 액션이 포함된 공통 CRUD 화면으로 구성했다.

관리자 로그인



운영자 대시보드

Revy Bank Admin

일반

대시보드

사용자 관리

어드민 관리

계좌 · 거래

계좌 관리

계좌이체

거래 내역

주식

종목 관리

포트폴리오

외환

통화

환율

환전

보험

보험 상품

로그아웃

대시보드

관리할 항목을 선택하세요.

어드민 관리

어드민 계정을 등록하고 관리합니다.

계정 관리

사용자 계정을 조회하고 관리합니다.

거래 내역

거래 기록을 조회합니다.

계좌 관리 - 대용량 계좌 데이터 그리드

Revy Bank Admin

일반

대시보드

사용자 관리

어드민 관리

계좌 · 거래

계좌 관리

계좌이체

거래 내역

주식

종목 관리

포트폴리오

외환

통화

환율

환전

보험

보험 상품

로그아웃

계좌 관리

+ 계좌 개설

사용자 ID

계좌번호

계좌유형

상태

예: 1

ACC-xxxx

REAL / VIRTUAL

ACTIVE / SUSPENDED / CL

통화

KRW / USD

Q 검색

초기화

ID	계좌번호	계좌명	사용자	유형	통화	잔고	가용잔고	상태	관리
187	ACC-DE...	예비 계좌	100	VIRTUAL	KRW	206580...	206580...	ACTIVE	상세 수정
186	ACC-DE...	주식 계좌	100	REAL	KRW	162605...	162605...	ACTIVE	상세 수정
185	ACC-DE...	예비 계좌	99	VIRTUAL	USD	995827...	995827...	ACTIVE	상세 수정
184	ACC-DE...	주식 계좌	99	REAL	USD	123530...	123530...	ACTIVE	상세 수정
183	ACC-DE...	주식 계좌	98	REAL	KRW	207650...	207650...	ACTIVE	상세 수정
182	ACC-DE...	주식 계좌	97	REAL	KRW	165104...	165104...	CLOSED	상세 수정
181	ACC-DE...	연금 계좌	96	REAL	KRW	577887...	577887...	ACTIVE	상세 수정
180	ACC-DE...	예비 계좌	96	VIRTUAL	KRW	500307...	500307...	ACTIVE	상세 수정

Page Size: 20

1 to 20 of 187

Page 1 of 10

종목 관리 - 증권 마스터 데이터 그리드

Revy Bank Admin

대시보드

사용자 관리

어드민 관리

계좌 · 거래

계좌 관리

계좌이체

거래 내역

주식

종목 관리

포트폴리오

외환

통화

환율

환전

보험

보험 상품

보험 증권

로그아웃

종목 관리

+ 종목 등록

티커

종목명

거래소

섹터

005930 / AAPL

삼성전자

KRX / NASDAQ / NYSE

IT

통화

KRW / USD

활성여부

true / false

Q 검색

초기화

ID	티커	종목명	거래소	섹터	통화	현재가	시가총액	활성	갱신시각
80	MRK	Merck &...	NYSE	Pharma...	USD	95	242000...	true	2026-0...
79	ABBV	AbbVie I...	NYSE	Pharma...	USD	195	344000...	true	2026-0...
78	LLY	Eli Lilly a...	NYSE	Pharma...	USD	900	860000...	true	2026-0...
77	CVX	Chevron...	NYSE	Energy	USD	155	283000...	true	2026-0...
76	WMT	Walmart...	NYSE	Retail	USD	95	765000...	true	2026-0...
75	KO	Coca-C...	NYSE	Beverages	USD	69	298000...	true	2026-0...
74	BAC	Bank of ...	NYSE	Banking	USD	45	348000...	true	2026-0...
73	HD	Home D...	NYSE	Retail	USD	410	408000...	true	2026-0...

Page Size: 20

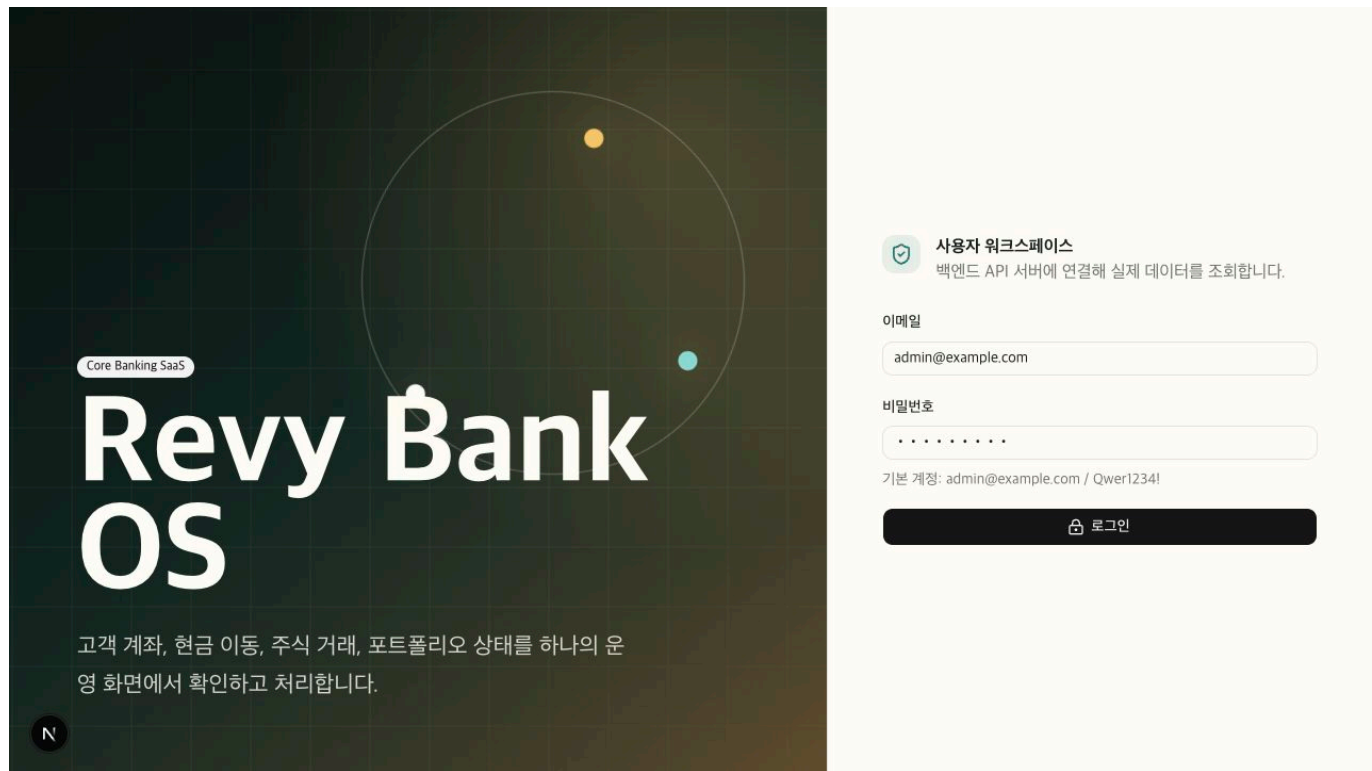
1 to 20 of 80

Page 1 of 4

프론트엔드 화면 - web-saas

사용자 워크스페이스는 계좌, 이체, 환전, 보험, 거래, 종목 조회를 하나의 금융 업무 셀에서 이동하도록 구성했다.

사용자 로그인



서비스 가입

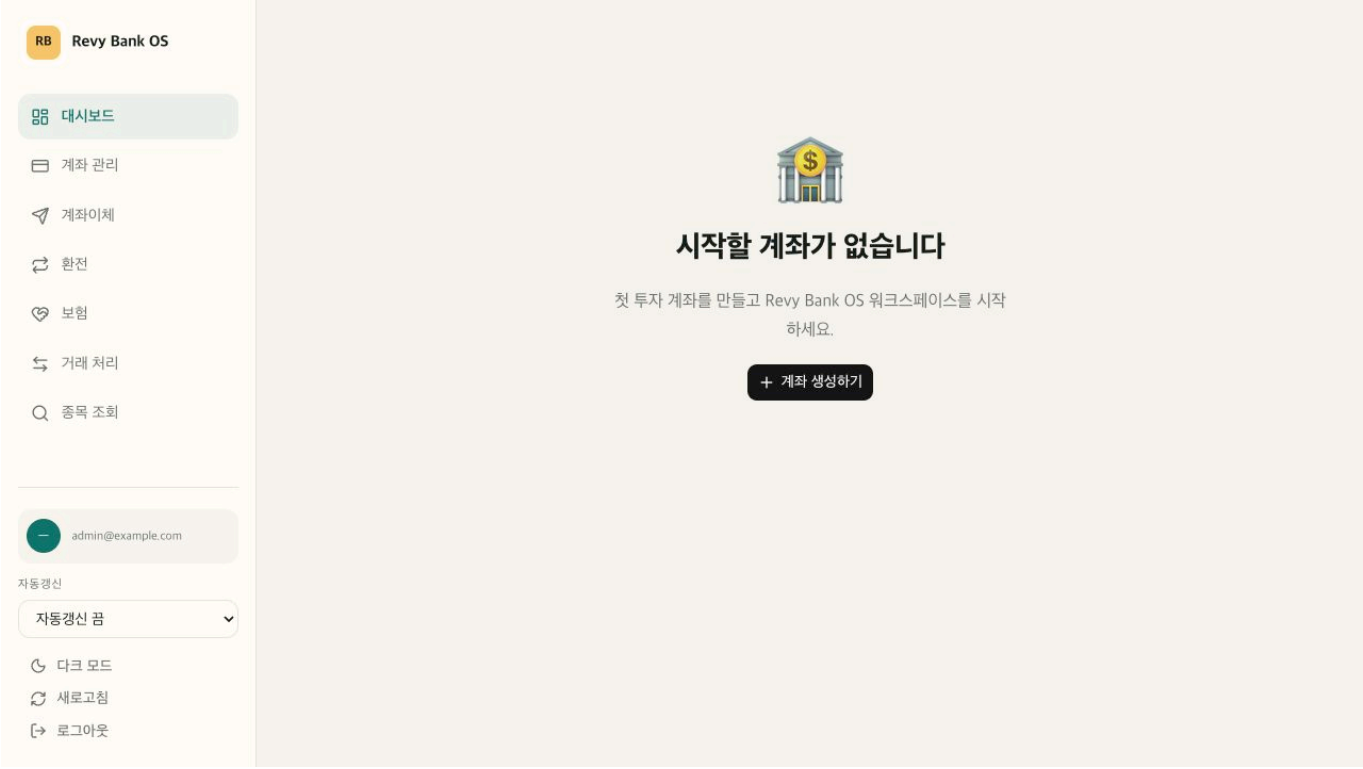
The image shows a service sign-up form titled '서비스 가입' (Service Sign-up) under the 'Core Banking SaaS' logo. The form is on a light yellow background with a subtle pattern. The title is in large black font, followed by the subtitle '사용자 SaaS 워크스페이스 생성을 위한 기본 정보를 입력하세요.' (Enter basic information for creating a user SaaS workspace).

The form fields are:

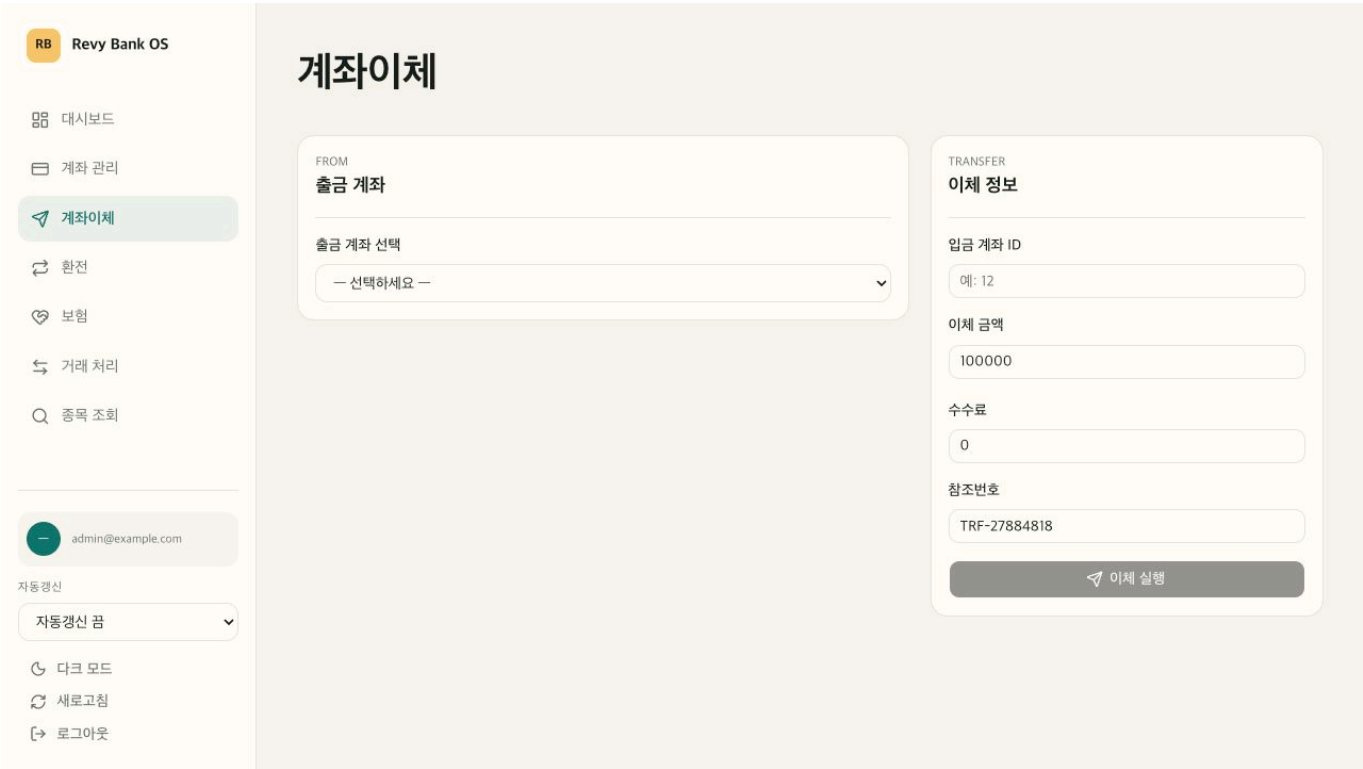
- 이름** (Name): Input field with '홍길동' (Hong Gildong).
- 이메일** (Email): Input field with 'user@example.com'.
- 회사 또는 조직명** (Company or Organization Name): Input field with 'Revy Capital'.
- 기본 통화** (Default Currency): Dropdown menu with 'KRW' selected.
- 비밀번호** (Password): Input field with a lock icon.

Below the password field, there's a note: '8자 이상, 영문과 숫자를 포함하세요.' (8 characters or more, including English letters and numbers). At the bottom, there's a label '비밀번호 확인' (Confirm Password).

워크스페이스 대시보드



계좌이체 업무 화면



INFRA

Grafana

Home

Bookmarks

Bookmark pages for them to appear here

Starred

Dashboards

Playlists

Snapshots

Library panels

Shared dashboards

Recently deleted

Explore

Drilldown

Alerting

Connections

Administration

Dashboards

Core Banking

Core Banking Operations

Job

monolith-api

Application

All

Instance

All

Prometheus

Last 15 minutes

Refresh

Share

JVM 스레드

Name	Last	Max
live - api-admin/local	27	27
live - api-saas/local	31	31
daemon - api-admin/local	17	17

CPU 사용률

Name	Last	Max
process - api-admin/local	0.302%	0.354%
process - api-saas/local	0.304%	0.357%
custom - api-admin/local	0.13%	0.13%

GC Pause p95

No data

DB 커넥션 풀

Name	Last	Max
active - HikariPool-1	0	0
active - HikariPool-1	0	0
idle - HikariPool-1	10	10
idle - HikariPool-1	10	10
pending - HikariPool-1	0	0
pending - HikariPool-1	0	0

애플리케이션 로그 이벤트

Name	Last	Max
debug	0 evt/s	0 evt/s
error	0 evt/s	0 evt/s
info	0 evt/s	0 evt/s
trace	0 evt/s	0 evt/s
warn	0 evt/s	0 evt/s

인스턴스별 메트릭 수집 지연

Name	Last	Max
api-admin/local	131 s	131 s
api-saas/local	5.02 s	5.03 s

JVM 클래스 / 파일 디스컴퍼

Name	Last	Max
loaded - api-admin/local	23.4 K	23.4 K
loaded - api-saas/local	22.3 K	22.3 K
open files - api-admin/local	198	198
open files - api-saas/local	186	186

Elastic
Find apps, content, and more.
AI Assistant

Untitled +

Data view All logs ▾

Search field names 0

- Available fields 18
- @timestamp
- _version
- APP_NAME
- clientip
- data_stream.dataset
- data_stream.namespace
- data_stream.type
- ENV
- level
- level_value
- logger_name
- LOGSTASH_ENABLED
- LOGSTASH_HOST
- message
- requesturl
- tags
- thread_name
- traceid
- Empty fields 6
- Meta fields 4

Filter your data using KQL syntax

Auto interval No breakdown

May 22, 2026 @ 14:42:00.503 - May 22, 2026 @ 14:42:10.232 (Interval: Auto - 194 milliseconds)

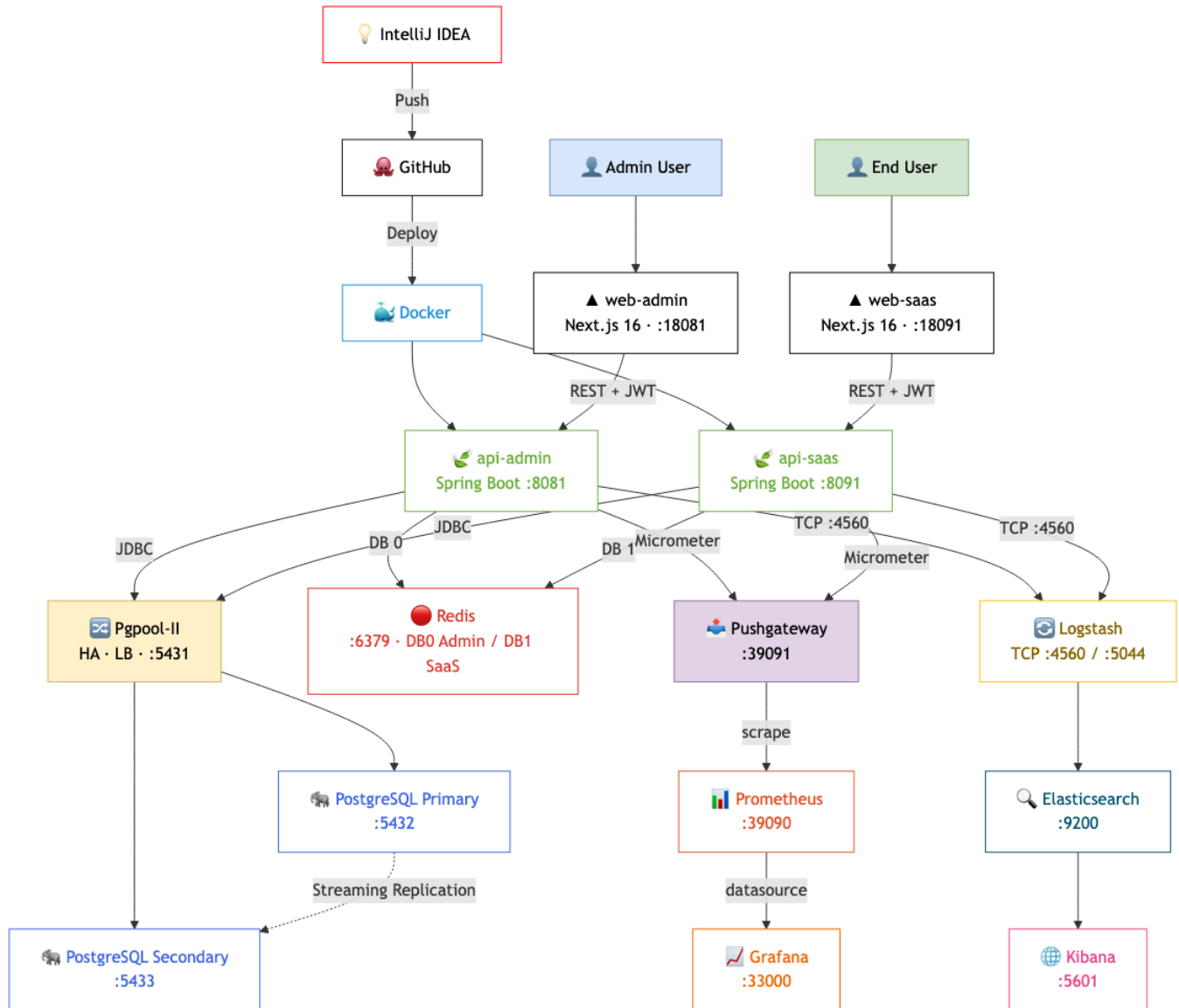
Check out context-aware Discover | Try Es|SQL | Inspect Alerts +

May 22, 2026 @ 14:42:00.5... → May 22, 2026 @ 14:42:10.232 Refresh

Documents (11) Field statistics

Summary

	@timestamp
May 22, 2026 @ 14:42:08.882	<pre> @timestamp May 22, 2026 @ 14:42:08.882 @version 1 APP_NAME api-admin clientip 39.115.90.140 data_stream.type logs ENV local level DEBUG level_value 10,000 logger_name org.hibernate.SQL LOGSTASH_ENABLED true LOGSTASH_HOST 127.0.0.1:4560 message select distinct a1.0_id, a1.0_created_at, a1.0_email, a1.0_name, a1.0_password, a1.0_role, r1.0.admin_id, r1.1.description, r1.1.name, r1.1.updated_at, a1.0.updated_at from admin a1 left join admin_roles r1_0 on a1.0.id=r1_0.admin_id left join admin_role r1_1 on r1.1.id=r1_0.role_ </pre>
May 22, 2026 @ 14:42:08.880	<pre> @timestamp May 22, 2026 @ 14:42:08.880 @version 1 APP_NAME api-admin clientip 39.115.90.140 data_stream.type logs ENV local level INFO level_value 20,000 logger_name com.rev.example.jwt.JwtSessionService LOGSTASH_ENABLED true LOGSTASH_HOST 127.0.0.1:4560 message principalType: ADMIN, claims:getSubject(): 6 requesturl /api/v1/auth/login thread_name tomcat-handler-6 traceId 019e4e34-922d-767e-b5ee-7994be8f436f _id AZSONOWkZzAaejqTVtVjg_ignored (null)_index .ds-logs-api-admin-local-2826.85.22-000001_score (null) </pre>
May 22, 2026 @ 14:42:08.782	<pre> @timestamp May 22, 2026 @ 14:42:08.782 @version 1 APP_NAME api-admin clientip 39.115.90.140 data_stream.type logs ENV local level DEBUG level_value 10,000 logger_name org.hibernate.SQL LOGSTASH_ENABLED true LOGSTASH_HOST 127.0.0.1:4560 message select pl.0.role_id, pl.0.permission from admin_role.permission pl.0 where pl.0.role_id=? requesturl /api/v1/auth/login n thread_name tomcat-handler-6 traceId 019e4e34-922d-767e-b5ee-7994be8f436f _id AZSONOWkZzAaejqTVtVjg_ignored (null)_index .ds-logs-api-admin-local-2826.85.22-000001_score (null) </pre>
May 22, 2026 @ 14:42:08.777	<pre> @timestamp May 22, 2026 @ 14:42:08.777 @version 1 APP_NAME api-admin clientip 39.115.90.140 data_stream.type logs ENV local level DEBUG level_value 10,000 logger_name org.hibernate.SQL LOGSTASH_ENABLED true LOGSTASH_HOST 127.0.0.1:4560 message select distinct a1.0_id, a1.0_created_at, a1.0_email, a1.0_name, a1.0_password, a1.0_role, r1.0.admin_id, r1.1.description, r1.1.name, r1.1.updated_at, a1.0.updated_at from admin a1 left join admin_roles r1_0 on a1.0.id=r1_0.admin_id left join admin_role r1_1 on r1.1.id=r1_0.role_ </pre>
May 22, 2026 @ 14:42:02.285	<pre> @timestamp May 22, 2026 @ 14:42:02.285 @version 1 APP_NAME api-admin clientip 39.115.90.140 data_stream.type logs ENV local level DEBUG level_value 10,000 logger_name org.hibernate.SQL LOGSTASH_ENABLED true LOGSTASH_HOST 127.0.0.1:4560 message select count(a1.0.id) from account a1.0 requesturl /api/v1/account thread_name tomcat-handler-6 traceId 019e4e34-775e-777b-9884-eefeb7b938e6 _id AZSONOWkZzAaejsPJT7fe_ignored (null)_index .ds-logs-api-admin-local-2826.85.22-000001_score (null) </pre>
May 22, 2026 @ 14:42:02.260	<pre> @timestamp May 22, 2026 @ 14:42:02.260 @version 1 APP_NAME api-admin clientip 39.115.90.140 data_stream.type logs ENV local level DEBUG level_value 10,000 logger_name org.hibernate.SQL LOGSTASH_ENABLED true LOGSTASH_HOST 127.0.0.1:4560 message select a1.0_id, a1.0.account_name, a1.0.account_number, a1.0.account_type, a1.0.available_balance, a1.0.balance, a1.0.create_date, a1.0.currency, a1.0.status, a1.0.updated_at, a1.0.user_id, a1.0.version from account a1.0 order by a1.0.id desc offset ? rows fetch first ? rows only requesturl /api/v1/accounts </pre>
May 22, 2026 @ 14:42:02.220	<pre> @timestamp May 22, 2026 @ 14:42:02.220 @version 1 APP_NAME api-admin clientip 39.115.90.140 data_stream.type logs ENV local level DEBUG level_value 10,000 logger_name org.hibernate.SQL LOGSTASH_ENABLED true LOGSTASH_HOST 127.0.0.1:4560 message select pl.0.role_id, pl.0.permission from admin_role.permission pl.0 where pl.0.role_id=? requesturl /api/v1/account t thread_name Tomcat-handler-8 traceId 019e4e34-775e-777b-9884-eefeb7b938e6 _id AZSONOWkZzAaejsPJT7fe_ignored (null)_index .ds-logs-api-admin-local-2826.85.22-000001_score (null) </pre>



백엔드 API 문서

관리자/사용자 API 스크린샷은 저장된 OpenAPI JSON과 컨트롤러 기준 API 목록을 문서용 이미지로 렌더링했다. 전체 API 목록은 아래 표에 컨트롤러 기준으로 보강했다.

BACKEND ADMIN API

ADMIN-API OpenAPI

ADMIN-API 명세서 · 10 paths

Method	Path	Operation	Tag
POST	/api/v1/auth/refresh	refresh	auth-controller
POST	/api/v1/auth/logout	logout	auth-controller
POST	/api/v1/auth/login	login	auth-controller
GET	/api/v1/admin	search	admin-controller
POST	/api/v1/admin	create	admin-controller
GET	/api/v1/accounttransaction	search_1	account-transaction-controller
POST	/api/v1/accounttransaction	create_1	account-transaction-controller
GET	/api/v1/account	search_2	account-controller
POST	/api/v1/account	create_2	account-controller
GET	/api/v1/admin/{id}	get	admin-controller
PATCH	/api/v1/admin/{id}	update	admin-controller
DELETE	/api/v1/admin/{id}	delete	admin-controller
GET	/api/v1/accounttransaction/{id}	get_1	account-transaction-controller
PATCH	/api/v1/accounttransaction/{id}	update_1	account-transaction-controller
DELETE	/api/v1/accounttransaction/{id}	delete_1	account-transaction-controller
GET	/api/v1/account/{id}	get_2	account-controller
PATCH	/api/v1/account/{id}	update_2	account-controller

BACKEND SAAS API

SAAS-API OpenAPI

SAAS-API 명세서 · 18 paths

Method	Path	Operation	Tag
POST	/api/v1/auth/refresh	refresh	auth-controller
POST	/api/v1/auth/logout	logout	auth-controller
POST	/api/v1/auth/login	login	auth-controller
GET	/api/v1/accounts	search	account-controller
POST	/api/v1/accounts	create	account-controller
POST	/api/v1/accounts/{accountId}/withdraw	withdraw	account-controller
POST	/api/v1/accounts/{accountId}/trades/sell	sell	trade-controller
POST	/api/v1/accounts/{accountId}/trades/dividend	dividend	trade-controller
POST	/api/v1/accounts/{accountId}/trades/buy	buy	trade-controller
POST	/api/v1/accounts/{accountId}/deposit	deposit	account-controller
GET	/api/v1/accounts/{accountId}	get	account-controller
PATCH	/api/v1/accounts/{accountId}	update	account-controller
DELETE	/api/v1/accounts/{accountId}	close	account-controller
GET	/api/v1/stocks	search_1	stock-controller
GET	/api/v1/stocks/{stockId}	get_1	stock-controller
GET	/api/v1/auth/me	me	auth-controller
GET	/api/v1/accounts/{accountId}/transactions	searchTransactions	account-controller

2. 백엔드 모듈 구조

```

backend/
├── module/
│   ├── core/
│   │   ├── common          # BigDecimalUtil, UuidUtil 등 공통 utils
│   │   ├── core-domain     # BaseEntity
│   │   ├── core-exception  # BusinessException, ErrorCode
│   │   └── core-web        # 예외 처리, 웹 공통 설정
│   ├── domain              # Account, User, Stock, Order, FX, Insurance,
│   │   Ledger, Billing 엔티티
│   ├── business-logic      # Reader/Command 기반 도메인 유스케이스 구현
│   ├── jwt-auth            # JWT 발급, 검증, principal 처리
│   └── quartz-batch        # Quartz / Spring Batch 공통·관리·실행 모듈

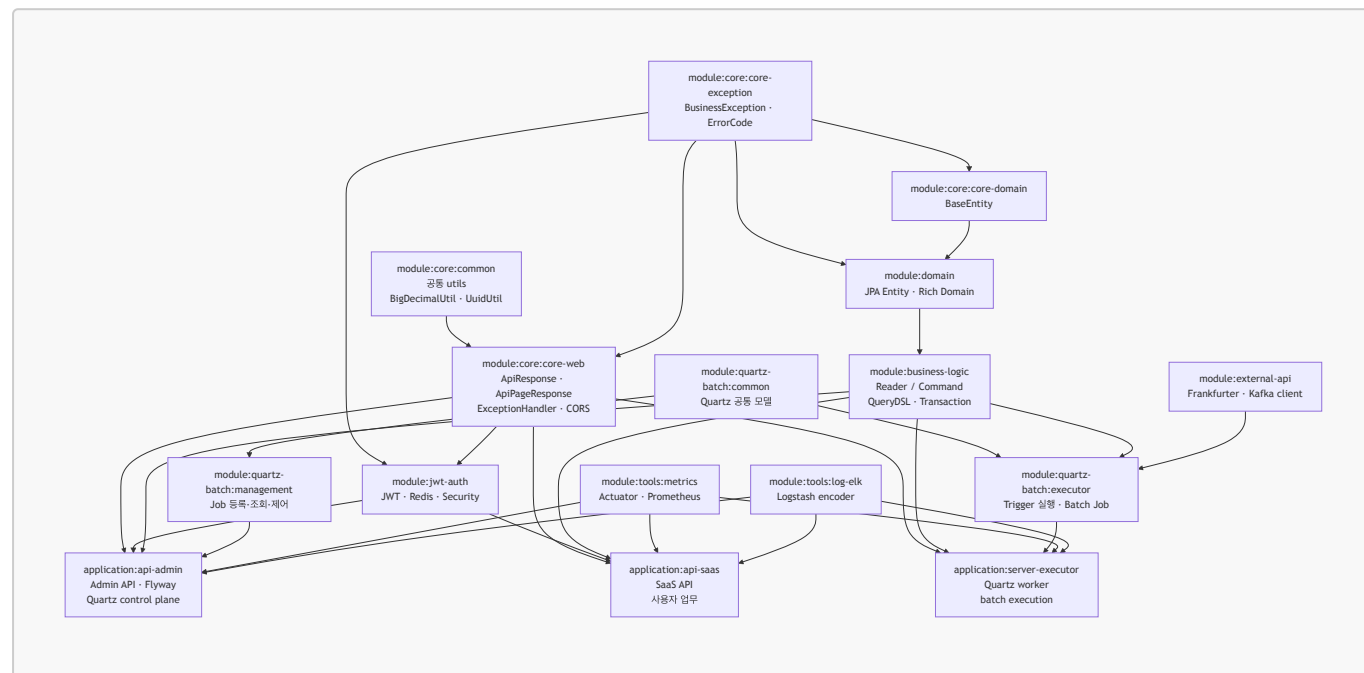
```

```

├── external-api      # Frankfurter 환율 API, Kafka client
├── tools             # metrics, log-elk
└── application/
    ├── api-admin     # 관리자 API, 기본 포트 8081
    ├── api-saas      # 사용자 API, 기본 포트 8091
    └── server-executor # Quartz Trigger 실행 워커, 기본 포트 8071

```

아래 다이어그램의 화살표는 의존 대상 -> 사용하는 모듈 방향이다.



실제 Gradle 의존성

모듈	직접 의존 모듈	역할
<code>module:core:common</code>	없음	공통 utils. <code>BigDecimalUtil</code> , <code>UuidUtil</code>
<code>module:core:core-exception</code>	없음	업무 예외와 에러 코드
<code>module:core:core-domain</code>	<code>core-exception</code>	<code>BaseEntity</code> 와 엔티티 공통 기반
<code>module:core:core-web</code>	<code>core-common</code> , <code>core-exception</code>	공통 응답, 전역 예외 처리, CORS/OpenAPI/Validation 공통
<code>module:domain</code>	<code>core-exception</code> , <code>core-domain</code>	JPA 엔티티, enum, 도메인 예외, Rich Domain 로직
<code>module:business-logic</code>	<code>domain</code>	QueryDSL Reader, 트랜잭션 Command, DTO 경계
<code>module:jwt-auth</code>	<code>core-exception</code> , <code>core-web</code>	JWT 발급/검증, Redis refresh token, Security 필터 지원
<code>module:quartz-batch:quartz-batch-</code>	없음	Quartz Job/Trigger 공통 모델과 상태 코드

모듈	직접 의존 모듈	역할
common		
module:quartz-batch:quartz-batch-management	quartz-batch-common	Job 등록·수정·삭제, 실행 이력 조회, pause/resume/runNow 제어
module:quartz-batch:quartz-batch-executor	quartz-batch-common, business-logic, external-api:frankfurter-client	Trigger 발화, Batch Job 실행, 외부 환율 수집 작업
module:external-api:frankfurter-client	없음	Frankfurter 환율 API RestClient
module:external-api:kafka-client	없음	Kafka producer/consumer 공통 설정 후보 모듈
module:tools:metrics	없음	Actuator, Prometheus, Pushgateway 연동
module:tools:log-elk	없음	Logstash encoder 기반 로그 전송
application:api-admin	core-web, jwt-auth, business-logic, quartz-batch-management, metrics, log-elk	운영자 API, Flyway, 운영/정산/Quartz 제어 조립
application:api-saas	core-web, jwt-auth, business-logic, metrics, log-elk	사용자 API와 본인 계좌 중심 업무 조립
application:server-executor	core-web, business-logic, quartz-batch-executor, metrics, log-elk	Quartz Trigger 실행과 Batch Job 처리를 담당하는 워커

백엔드 설계 포인트

영역	설계
계층 분리	Application Controller는 payload/usecase 중심, 도메인 엔티티 직접 노출 최소화
조회/변경 분리	Reader는 조회, Command는 상태 변경과 트랜잭션 책임
인증	Admin/SaaS 별도 JWT issuer와 Redis DB 사용
데이터 정합성	계좌 입출금/이체, 주식 매매, 환전, 보험료 납부는 단일 트랜잭션에서 처리
감사 추적	referenceId, 거래 내역, 원장 분개로 업무 이벤트 추적
운영 기능	청구/정산 상태 전이, RBAC, Quartz/Spring Batch 운영 API 제공

도메인 분석 요약

도메인	구현 관점	포트폴리오에서 보여줄 포인트
계좌/거래	잔고·가용잔고 분리, 입출금/이체 거래 기록, 낙관적 락	동시성, 멍등성, 소유권 검증
주식/주문/포지션	주문 접수·체결, 매수 Lot, FIFO 매도, 실현손익 계산	단순 CRUD를 넘어선 상태 전이와 자산 평가
외환	통화/환율 마스터, 환전 요청·완료, 계좌 출금/입금 연결	크로스 도메인 트랜잭션
보험	상품, 증권, 수익자, 보험료, 청구 심사/지급	긴 생명주기의 업무 상태 모델링
원장	계정과목, 회계기간, 분개, 역분개, 시산표	복식부기 균형 검증과 감사 추적
청구/정산	청구서 발행·결제·연체, 정산 성공/실패	운영 백오피스 업무 흐름
RBAC/스케줄러	관리자 역할·권한, Quartz/Batch 조회·제어	운영자 콘솔의 실무 기능

3. 프론트엔드 디자인 구조

```
frontned/  
├── web-admin/  
│   └── src/  
│       ├── app/(auth)/login  
│       ├── app/(dashboard)/dashboard  
│       ├── components/data-grid      # CrudPage, DataGrid, Modal, SearchBar  
│       ├── features                  # 도메인별 CRUD config  
│       ├── lib                       # API client, auth store  
│       └── services                  # CRUD service  
└── web-saas/  
    └── src/  
        ├── app/login, signup  
        ├── app/workspace             # 계좌, 이체, 주식, 거래, 외환, 보험  
        ├── features                  # 도메인별 service/type/component  
        ├── shared                    # API client, auth token, 공통 타입  
        └── workspace                 # Sidebar, TopNav, Provider
```

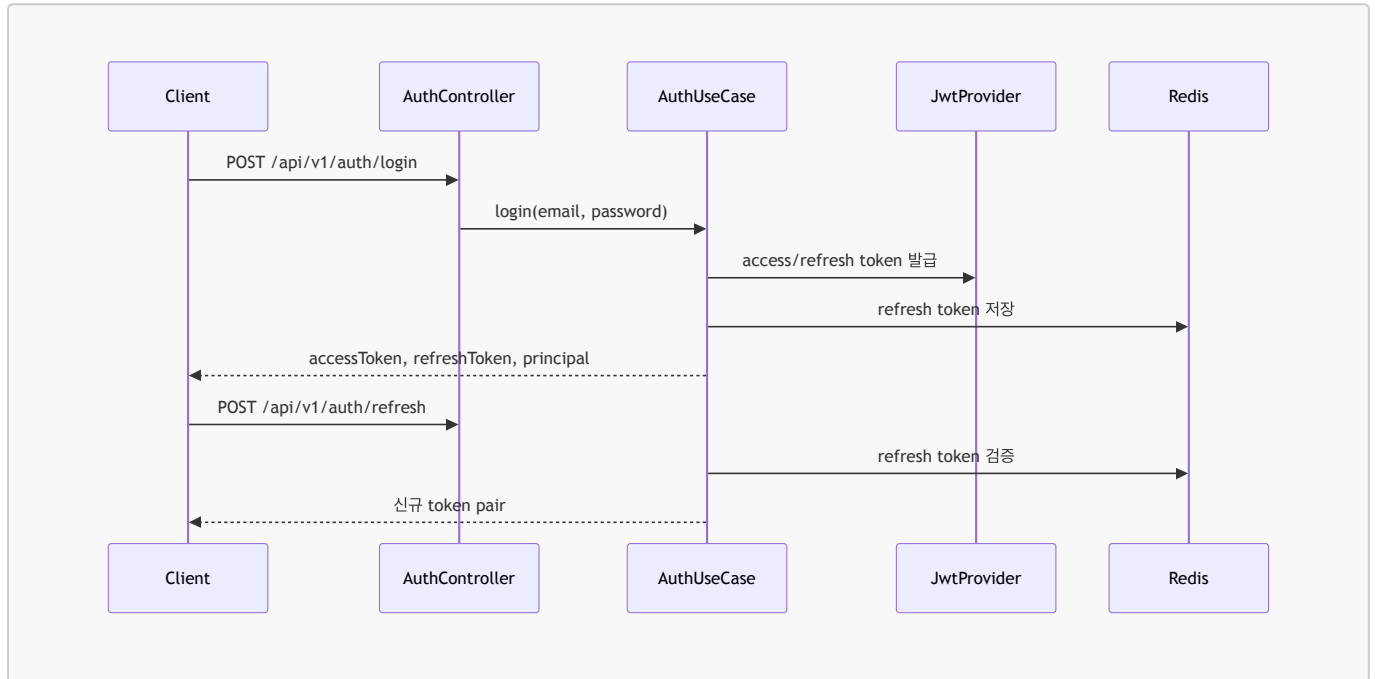
화면 설계

앱	핵심 화면	목적
web-admin	Dashboard	운영 지표와 관리 메뉴 진입
web-admin	계좌/거래/사용자/관리자	서버사이드 페이지네이션 기반 CRUD 관리
web-admin	FX/보험/원장/청구/정산/스케줄러	환율, 보험 증권/청구, 계정과목/분개/시산표, 청구서, 정산, 배치 운영
web-saas	Workspace	사용자의 금융 활동 허브

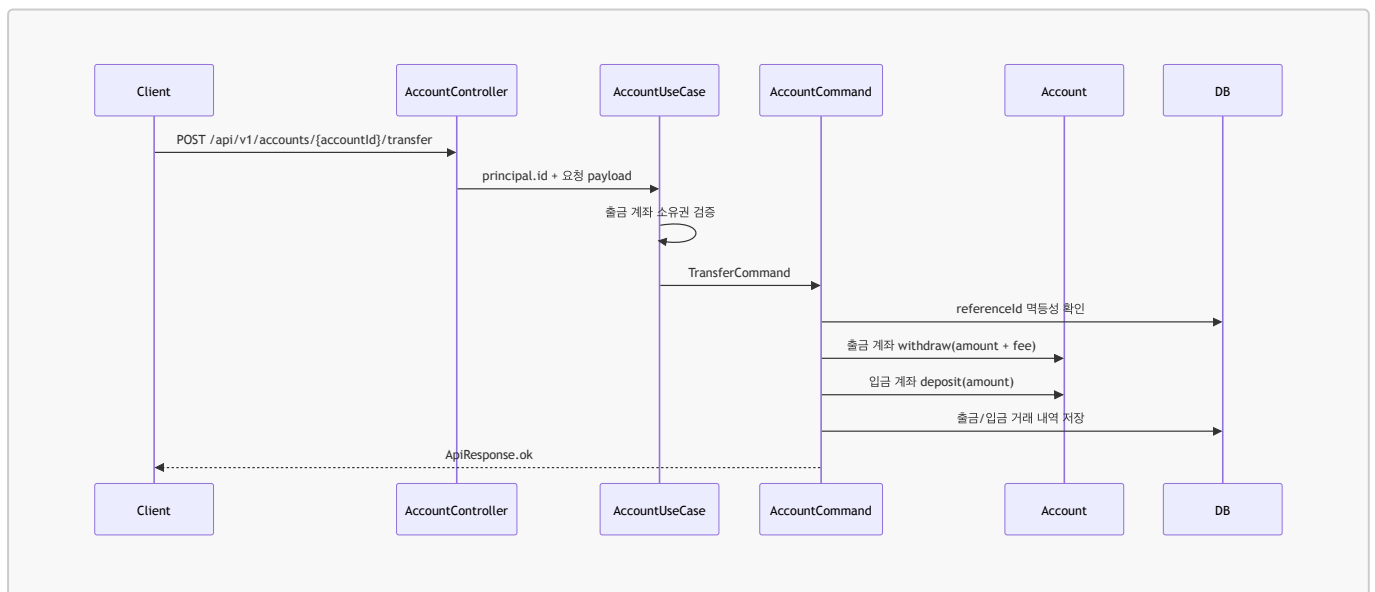
앱	핵심 화면	목적
web-saas	Accounts/Transfer	계좌 개설, 잔고 확인, 입출금, 이체
web-saas	Stocks/Trades/Portfolio	종목 조회, 매매, 포트폴리오 평가
web-saas	FX/Insurance/Billing	환전, 보험 상품 조회, 가입, 청구, 청구서 결제

4. 백엔드 플로우

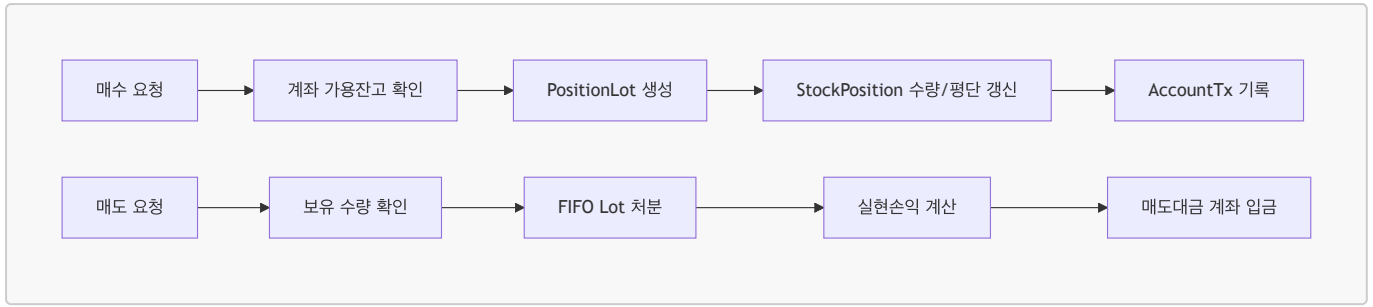
인증 플로우



계좌 이체 플로우

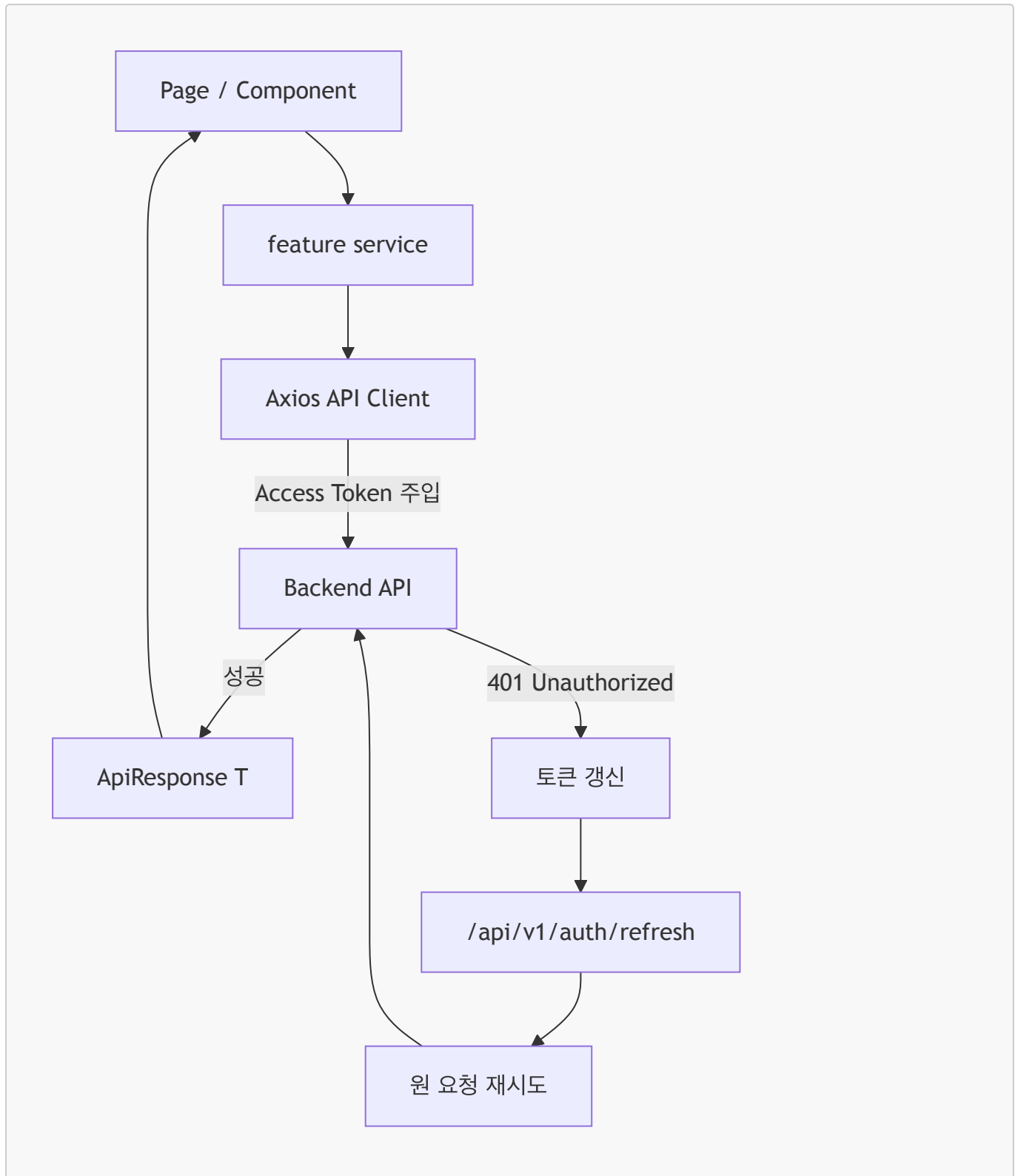


주식 매매 플로우

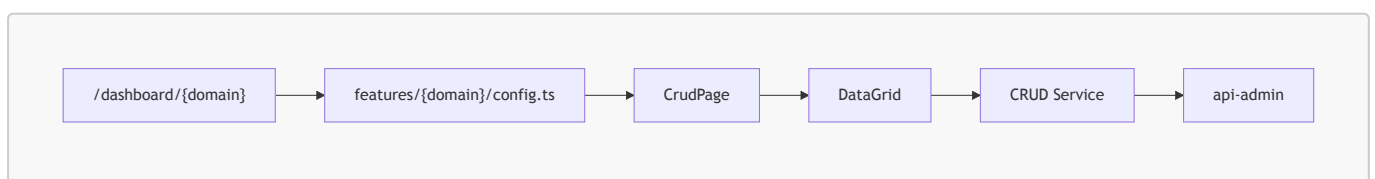


5. 프론트엔드 플로우

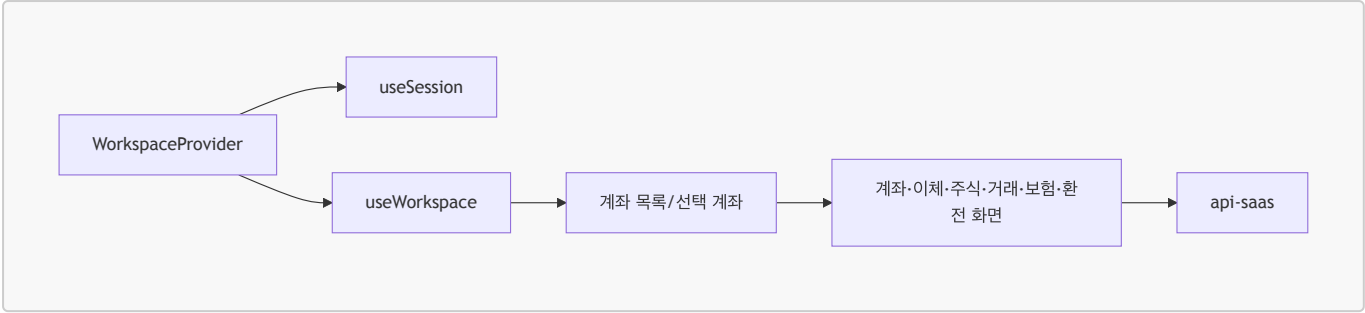
공통 API 호출 흐름



web-admin CRUD 흐름



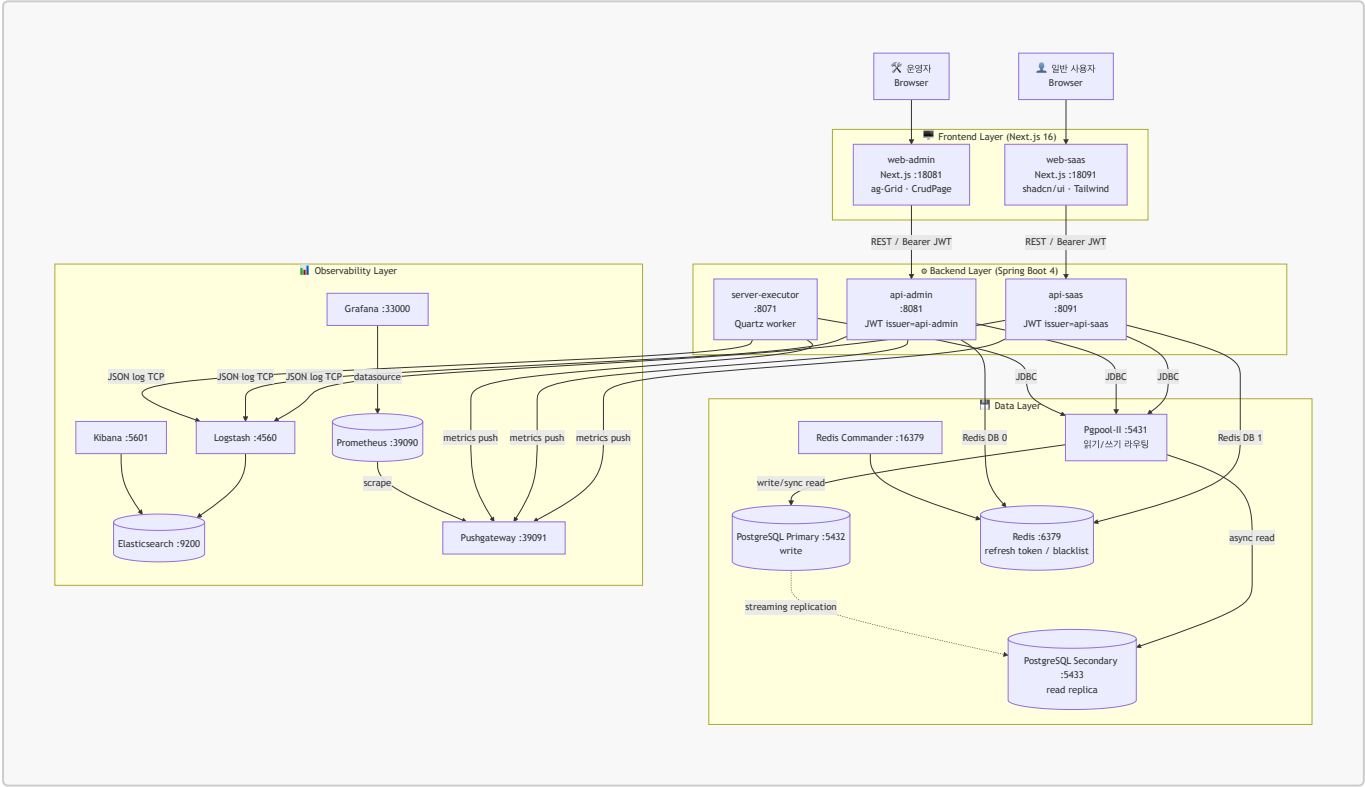
web-saas 워크스페이스 흐름



6. 인프라 구성도

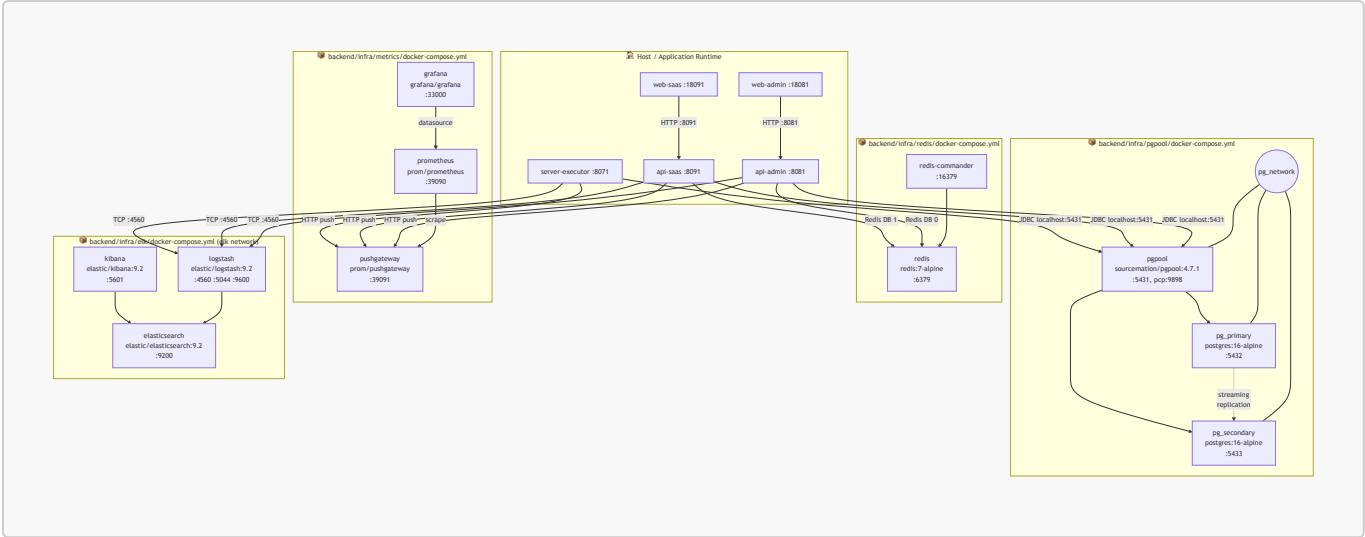
이 프로젝트는 로컬 **Docker Compose** 기반으로 운영 환경을 재현한다. 프론트엔드(Next.js 2개) → 백엔드(Spring Boot API 2개 + Executor 1개) → 데이터 계층(pgpool+PostgreSQL HA, Redis) → 관측성 스택(ELK, Prometheus+Grafana)으로 구성된다. 4개의 Docker Compose 스택이 독립적으로 기동되어 필요한 영역만 선택적으로 실행할 수 있다.

6.1 전체 시스템 아키텍처 (논리 계층)



6.2 Docker Compose 스택 구성

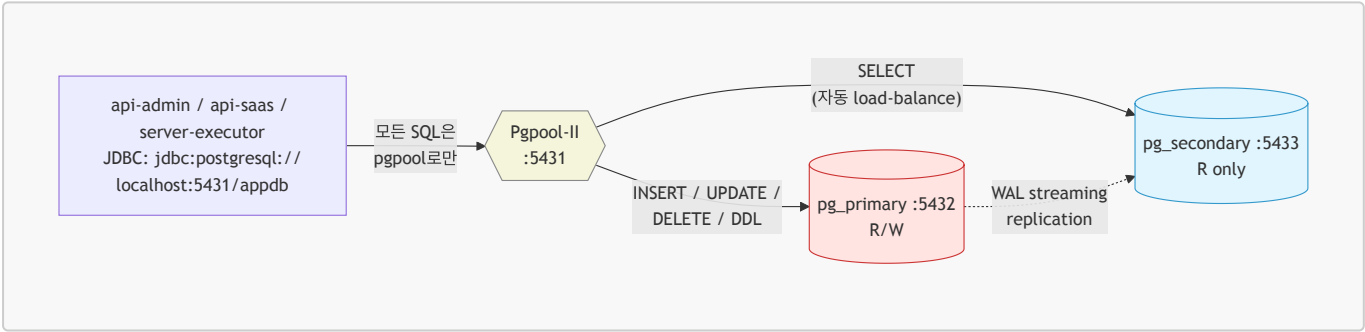
4개의 독립적인 Docker Compose 파일이 인프라를 구성한다. 애플리케이션 코드(**api-admin**, **api-saas**, **server-executor**, **web-admin**, **web-saas**)는 호스트(또는 별도 컨테이너)에서 실행되어 도커 네트워크 외부에서 접근한다.



기동/중지 통합 스크립트 — `script/all-start.sh` → 4개 인프라 compose 기동 → 백엔드 API 2개와 Executor 1개 부팅 → 프론트엔드 2개 빌드·서빙

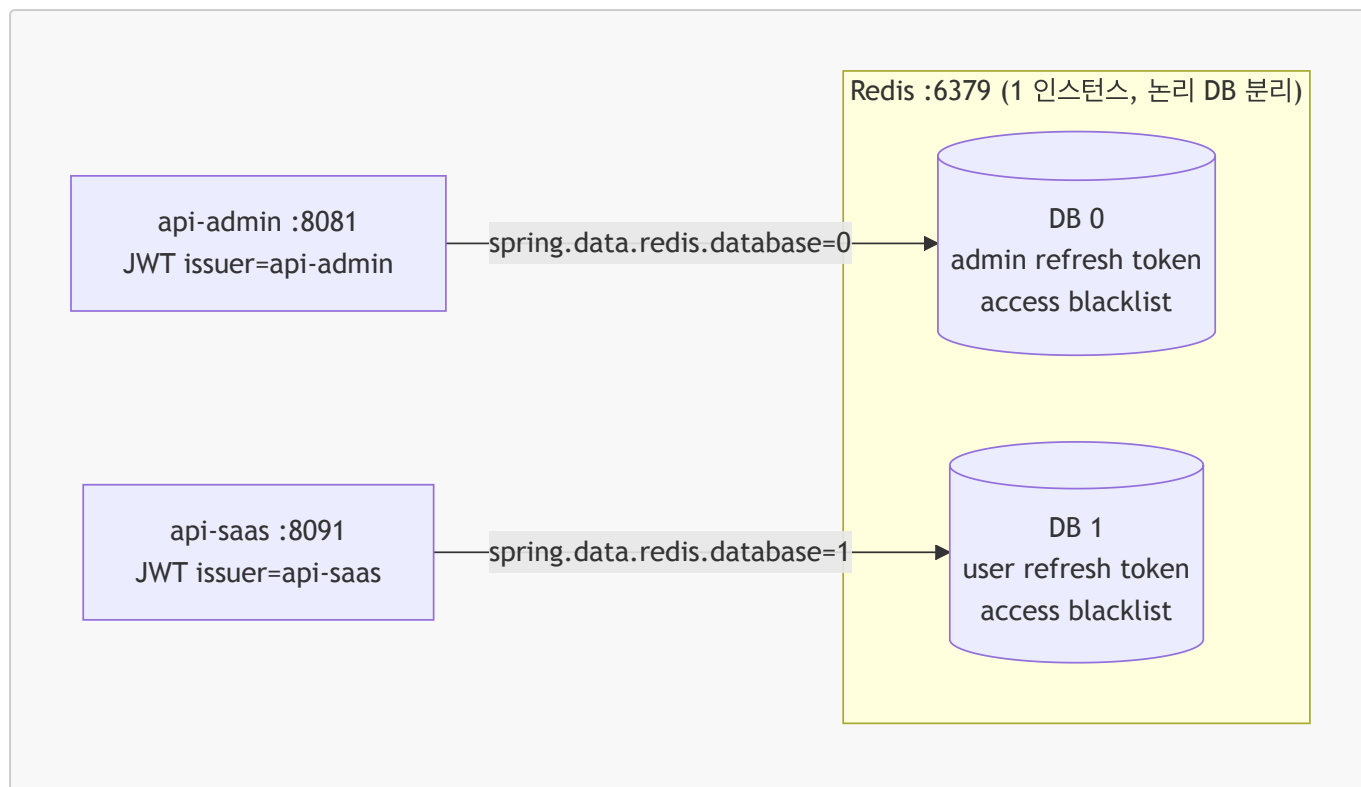
6.3 데이터 영속화 토폴로지 (Pgpool-II HA)

코어뱅킹 특성상 데이터 가용성이 중요하므로, PostgreSQL Primary-Secondary 복제 + Pgpool-II를 통한 자동 read/write 라우팅을 구성했다.



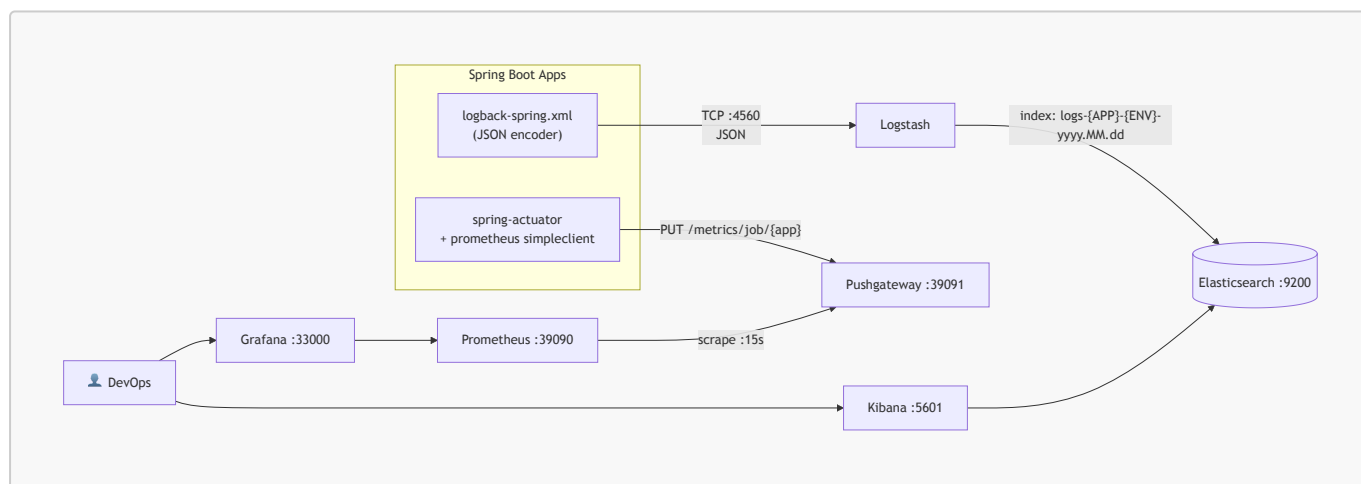
- 앱은 **pgpool**만 바라본다 (`jdbc:postgresql://localhost:5431/appdb`). 노드 장애·교체 시에도 connection string 변경 불필요.
- **Flyway**는 **primary**에 직접 작성 (pgpool 경유). DDL은 자동으로 secondary로 streaming replication.
- 운영 환경 적용 시: Pgpool watchdog + HAProxy + replication lag 모니터링까지 확장 예정.

6.4 인증 / 세션 토폴로지 (Redis 분리)



- **JWT issuer 분리:** 관리자 토큰이 사용자 API에서 검증되지 않도록 issuer claim을 다르게 한다.
- **Redis DB 분리:** 같은 Redis 인스턴스에서도 키 충돌·운영 실수 방지를 위해 DB index를 분리.
- **logout/refresh:** refresh token은 Redis에 저장, access token blacklist도 Redis로 즉시 무효화.

6.5 관측성 데이터 흐름



- **로그:** Logback이 logstash-encoder로 JSON 직렬화 → TCP appender로 Logstash 전송 → Elasticsearch 인덱싱.
- **메트릭:** 단명(short-lived) 작업도 누락 없이 수집하기 위해 Pushgateway 방식을 채택. Prometheus는 Pushgateway만 scrape.
- **시각화:** Kibana(로그 검색·분석), Grafana(시계열·SLO 대시보드)로 역할 분리.

6.6 포트 맵 (Port Map)

분류	컴포넌트	노출 포트	컨테이너 포트	비고
Frontend	web-admin (Next.js)	18081	—	운영자 콘솔
	web-saas (Next.js)	18091	—	사용자 워크스페이스
Backend	api-admin (Spring Boot)	8081	—	운영자 API, Swagger /swagger-ui/index.html
	api-saas (Spring Boot)	8091	—	사용자 API, Swagger /swagger-ui/index.html
	server-executor (Spring Boot)	8071	—	Quartz Trigger 실행 워커
Database	pgpool	5431	5432	앱 단일 진입점
	pg_primary	5432	5432	쓰기 노드
	pg_secondary	5433	5432	읽기 복제
	pgpool pcp	9898	9898	pgpool 관리 CLI
Cache	redis	6379	6379	DB0=admin · DB1=user
	redis-commander	16379	8081	Redis 관리 UI
Logging	logstash (TCP)	4560	4560	앱 로그 입력
	logstash (Beats)	5044	5044	(선택)
	logstash (Monitoring)	9600	9600	API
	elasticsearch	9200	9200	로그 저장소
	kibana	5601	5601	로그 시각화
Metrics	pushgateway	39091	9091	메트릭 push
	prometheus	39090	9090	메트릭 수집
	grafana	33000	3000	대시보드 (admin/admin)

6.7 Compose 파일 ↔ 컨테이너 매핑

Docker Compose 파일	네트워크	컨테이너
backend/infra/pgpool/docker-compose.yml	pg_network	pg_primary , pg_secondary , pgpool
backend/infra/redis/docker-compose.yml	default	redis , redis-commander
backend/infra/metrics/docker-compose.yml	default	pushgateway , prometheus , grafana

Docker Compose 파일	네트워크	컨테이너
backend/infra/elk/docker-compose.yml	elk	elasticsearch, logstash, kibana

6.8 네트워크 / 운영 설계 포인트

- **DB 단일 진입점:** 애플리케이션은 PostgreSQL primary/secondary에 직접 접속하지 않고 **pgpool:5431**만 사용한다. → 노드 교체 시 connection string 변경 불요.
- **Reader/Command 분리 + Pgpool:** 향후 hint(**/*NO LOAD BALANCE***/)나 connection routing으로 **Reader** 트랜잭션을 secondary 노드로 강제 라우팅 가능.
- **인증 격리:** Admin/SaaS는 단일 Redis 인스턴스를 공유하되 DB index를 **0, 1**로 분리. JWT issuer claim도 다르게 설정해 cross-app token 사용을 차단.
- **로그 일관성:** Spring **logback-spring.xml**이 JSON encoder로 직렬화하여 Logstash TCP input(**4560**)으로 전송. 인덱스 패턴은 **logs-{APP_NAME}-{ENV}-yyyy.MM.dd**로 앱·환경·일자 단위 검색 가능.
- **메트릭 수집 방식:** Spring Actuator + Prometheus simpleclient가 Pushgateway에 push → Prometheus가 Pushgateway를 scrape. 배치성 작업도 메트릭 누락 없음.
- **통합 기동 스크립트:** **script/all-start.sh** 한 번으로 4개 인프라 스택 → 백엔드 API 2개와 Executor 1개 → 프론트엔드 2개 순차 기동. 종료는 **all-stop.sh**, 재시작은 **all-restart.sh**.
- **헬스체크:** 모든 핵심 컨테이너(**pg_primary, pg_secondary, pgpool, redis**)에 **healthcheck** 정의 → **depends_on.condition: service_healthy**로 기동 순서 보장.

7. 백엔드 API 목록

api-admin

Method	Path	설명
POST	/api/v1/auth/login	관리자 로그인
POST	/api/v1/auth/refresh	관리자 토큰 재발급
POST	/api/v1/auth/logout	관리자 로그아웃
GET	/api/v1/auth/me	관리자 principal 조회
GET	/api/v1/admin	관리자 목록 조회
POST	/api/v1/admin	관리자 생성 엔드포인트, 현재 구현은 제한적
GET	/api/v1/admin/{id}	관리자 단건 조회
PATCH	/api/v1/admin/{id}	관리자 수정 엔드포인트, 현재 구현은 제한적
DELETE	/api/v1/admin/{id}	관리자 삭제 엔드포인트, 현재 구현은 제한적
GET	/api/v1/user	사용자 목록 조회
GET	/api/v1/user/{id}	사용자 단건 조회

Method	Path	설명
PATCH	/api/v1/user/{id}	사용자 수정 엔드포인트, 현재 구현은 제한적
GET	/api/v1/account	계좌 목록 조회
POST	/api/v1/account	계좌 개설
GET	/api/v1/account/{id}	계좌 단건 조회
PATCH	/api/v1/account/{id}	계좌명 수정
DELETE	/api/v1/account/{id}	계좌 폐쇄
POST	/api/v1/account/{id}/suspend	계좌 정지
POST	/api/v1/account/{id}/deposit	계좌 입금
POST	/api/v1/account/{id}/withdraw	계좌 출금
POST	/api/v1/account/{id}/transfer	계좌 이체
GET	/api/v1/account_tx	계좌 거래 내역 목록 조회
GET	/api/v1/account_tx/{id}	계좌 거래 내역 단건 조회
GET	/api/v1/stock	종목 목록 조회
POST	/api/v1/stock	종목 등록
GET	/api/v1/stock/{id}	종목 단건 조회
DELETE	/api/v1/stock/{id}	종목 상장폐지
PATCH	/api/v1/stock/{id}/market-data	종목 시장 데이터 수정
GET	/api/v1/portfolio/{accountId}	계좌별 포트폴리오 조회
POST	/api/v1/fx/currency	통화 등록
GET	/api/v1/fx/currency	활성 통화 목록 조회
GET	/api/v1/fx/currency/{code}	통화 단건 조회
POST	/api/v1/fx/currency/{code}/activate	통화 활성화
POST	/api/v1/fx/currency/{code}/deactivate	통화 비활성화
POST	/api/v1/fx/rate	환율 등록
GET	/api/v1/fx/rate	환율 목록 조회
GET	/api/v1/fx/rate/current	통화쌍별 현재 환율 조회
GET	/api/v1/fx/rate/history	환율 변경 이력 조회

Method	Path	설명
GET	/api/v1/fx/corridor	통화 회랑 목록 조회
POST	/api/v1/fx/corridor	통화 회랑 등록
PATCH	/api/v1/fx/corridor/{id}	통화 회랑 한도·스프레드 수정
DELETE	/api/v1/fx/corridor/{id}	통화 회랑 삭제
POST	/api/v1/fx/corridor/{id}/activate	통화 회랑 활성화
POST	/api/v1/fx/corridor/{id}/deactivate	통화 회랑 비활성화
POST	/api/v1/fx/corridor/{id}/suspend	통화 회랑 정지
POST	/api/v1/fx/conversion	환전 실행
GET	/api/v1/fx/conversion/{id}	환전 내역 단건 조회
GET	/api/v1/insurance/product	보험 상품 목록 조회
POST	/api/v1/insurance/product	보험 상품 등록
GET	/api/v1/insurance/product/{id}	보험 상품 단건 조회
PATCH	/api/v1/insurance/product/{id}	보험 상품 가격 수정
DELETE	/api/v1/insurance/product/{id}	보험 상품 판매 중단
POST	/api/v1/insurance/product/{id}/discontinue	보험 상품 판매 중단
POST	/api/v1/insurance/policy	보험 증권 발행
GET	/api/v1/insurance/policy	보험 증권 목록 조회
GET	/api/v1/insurance/policy/{id}	보험 증권 단건 조회
POST	/api/v1/insurance/policy/{id}/activate	보험 증권 활성화
POST	/api/v1/insurance/policy/{id}/suspend	보험 증권 정지
POST	/api/v1/insurance/policy/{id}/reactivate	보험 증권 재활성화
POST	/api/v1/insurance/policy/{id}/terminate	보험 증권 만료 처리
POST	/api/v1/insurance/policy/{id}/cancel	보험 증권 취소
POST	/api/v1/insurance/policy/{id}/pay-premium	보험료 납부 실행
POST	/api/v1/insurance/policy/payment/{paymentId}/overdue	보험료 연체 처리
POST	/api/v1/insurance/claim	보험금 청구 접수
GET	/api/v1/insurance/claim	보험금 청구 목록 조회
GET	/api/v1/insurance/claim/{id}	보험금 청구 단건 조회
POST	/api/v1/insurance/claim/{id}/start-review	보험금 청구 심사 시작

Method	Path	설명
POST	/api/v1/insurance/claim/{id}/approve	보험금 청구 승인
POST	/api/v1/insurance/claim/{id}/reject	보험금 청구 거절
POST	/api/v1/insurance/claim/{id}/pay	보험금 지급 실행
GET	/api/v1/ledger/account	계정과목 목록 조회
POST	/api/v1/ledger/account	계정과목 생성
GET	/api/v1/ledger/account/{id}	계정과목 단건 조회
PATCH	/api/v1/ledger/account/{id}	계정과목명 변경
DELETE	/api/v1/ledger/account/{id}	계정과목 사용 중단
POST	/api/v1/ledger/account/{id}/discontinue	계정과목 사용 중단
POST	/api/v1/ledger/period	회계기간 개설
GET	/api/v1/ledger/period/{id}	회계기간 단건 조회
POST	/api/v1/ledger/period/{id}/close	회계기간 마감
POST	/api/v1/ledger/journal	분개 생성 및 전기
GET	/api/v1/ledger/journal	분개 목록 조회
GET	/api/v1/ledger/journal/{id}	분개 단건 조회
POST	/api/v1/ledger/journal/{id}/reverse	역분개 생성
GET	/api/v1/ledger/journal/trial-balance	시산표 조회

api-saas

Method	Path	설명
POST	/api/v1/auth/signup	사용자 회원가입
POST	/api/v1/auth/login	사용자 로그인
POST	/api/v1/auth/refresh	사용자 토큰 재발급
POST	/api/v1/auth/logout	사용자 로그아웃
GET	/api/v1/auth/me	사용자 principal 조회
POST	/api/v1/accounts	본인 계좌 개설
GET	/api/v1/accounts	본인 계좌 목록 조회

Method	Path	설명
GET	/api/v1/accounts/{accountId}	본인 계좌 단건 조회
PATCH	/api/v1/accounts/{accountId}	본인 계좌명 수정
DELETE	/api/v1/accounts/{accountId}	본인 계좌 폐쇄
POST	/api/v1/accounts/{accountId}/deposit	본인 계좌 입금
POST	/api/v1/accounts/{accountId}/withdraw	본인 계좌 출금
POST	/api/v1/accounts/{accountId}/transfer	본인 계좌 이체
GET	/api/v1/accounts/{accountId}/transactions	본인 계좌 거래 내역 조회
GET	/api/v1/accounts/{accountId}/transactions/{transactionId}	본인 거래 내역 단건 조회
GET	/api/v1/accounts/{accountId}/positions	본인 계좌 보유 종목 조회
GET	/api/v1/accounts/{accountId}/positions/{stockId}	본인 계좌 보유 종목 단건 조회
GET	/api/v1/accounts/{accountId}/portfolio	본인 계좌 포트폴리오 조회
GET	/api/v1/stocks	종목 목록 조회
GET	/api/v1/stocks/{stockId}	종목 단건 조회
POST	/api/v1/accounts/{accountId}/trades/buy	주식 매수
POST	/api/v1/accounts/{accountId}/trades/sell	주식 매도
POST	/api/v1/accounts/{accountId}/trades/dividend	배당금 입금
GET	/api/v1/fx/currencies	환전 가능 통화 목록 조회
GET	/api/v1/fx/rate/latest	최신 환율 조회
POST	/api/v1/fx/conversions	환전 실행
GET	/api/v1/fx/conversions/{id}	본인 환전 내역 단건 조회
GET	/api/v1/insurance/products	보험 상품 목록 조회
GET	/api/v1/insurance/products/{id}	보험 상품 단건 조회

Method	Path	설명
GET	/api/v1/insurance/policies	본인 보험 증권 목록 조회
POST	/api/v1/insurance/policies	보험 가입
GET	/api/v1/insurance/policies/{id}	본인 보험 증권 단건 조회
GET	/api/v1/insurance/claims	본인 보험금 청 구 목록 조회
POST	/api/v1/insurance/claims	보험금 청구 접 수
GET	/api/v1/insurance/claims/{id}	본인 보험금 청 구 단건 조회

8. 실행 정보

앱	URL	계정
api-admin Swagger	http://localhost:8081/swagger- ui/index.html	admin@example.com / Qwer1234!
api-saas Swagger	http://localhost:8091/swagger- ui/index.html	demo@example.com / Qwer1234!
web-admin	http://localhost:18081	admin@example.com / Qwer1234!
web-saas	http://localhost:18091	demo@example.com / Qwer1234!

9. 포트폴리오 강조 포인트

- 단순 CRUD가 아니라 계좌 잔고, 거래 내역, 주식 주문/포지션, 환전, 보험, 청구/정산, 복식부기 원장을 연결한 금융 도
메인 프로젝트다.
- Admin API와 SaaS API를 분리하여 운영자 권한과 사용자 권한 경계를 표현했다.
- 프론트엔드는 관리자 콘솔의 반복 CRUD 패턴과 사용자 워크스페이스의 금융 업무 흐름을 별도 UX로 설계했다.
- JWT silent refresh, Redis refresh token, QueryDSL 조회, Flyway migration, Quartz/Spring Batch 운영
API, Docker 기반 인프라 구성을 포함한다.
- PostgreSQL primary/secondary, Pgpool-II, Redis, ELK, Prometheus/Grafana까지 포함해 단일 앱 구현을
넘어 운영 관점의 인프라 설계를 표현했다.